for the rules-as-code projects like openfisca and policy engine, are there open formats that they use which can be ...

3:32 pm

17 sites

No universally standardized "open format" exists that allows you to seamlessly move executable legislative logic between different Rules-as-Code (RaC) platforms. Centre for Internation...

Because projects like OpenFisca and PolicyEngine (which originally forked from OpenFisca) rely on **procedural Python code** to execute formulas, calculations cannot simply be exported into a neutral configuration file (like JSON or YAML) and run elsewhere. Salsa Digital +4

However, interoperability exists at different layers of the RaC stack.

1. What You Can Move (Shared Parameters & Metadata)

While you cannot move the execution logic, both platforms use open, structured formats for the static components of law:

- **YAML Law Parameters**: OpenFisca and PolicyEngine isolate legislative values—such as tax brackets, eligibility age thresholds, or benefit amounts—into heavily structured **YAML files**. Because these are organized chronologically to map onto legislative amendments, you can easily parse and migrate these raw regulatory values between systems using standard scripts. PyPI +1
- **OpenAPI (JSON/YAML) Endpoints**: Both engines automatically expose their rules as REST APIs. They share a common interoperability layer using the OpenAPI Specification. A downstream application (like a chatbot or webform) can switch its backend from OpenFisca to PolicyEngine relatively cleanly because the data payloads can be formatted using open JSON Schemas. PyPI +2
- **HDF5 Data Ecosystem**: For microsimulation data, PolicyEngine leverages the open HDF5 (Hierarchical Data Format) standard to manage large national survey and demographic datasets. Any platform capable of parsing HDF5 can ingest these datasets. GitHub

2. The Architectural Divide

The core barrier to portability is that different RaC tools belong to entirely distinct computer science paradigms:

Feature / Paradigm Salsa Digital +5	OpenFisca & PolicyEngine	Blawx & DataLex
Primary Logic Type	**Imperative / Procedural** (Calculates *how* to find a numerical output using Python arrays)	**Declarative / Logic Programming** (Deductive reasoning based on facts and relationships)
Primary Use Case	Tax, benefits, microsimulation, and high-speed economic scaling	Legal compliance, statutory interpretation, and "yes/no" eligibility decisions
Underlying Language	Python code objects	Domain-Specific Languages (e.g., ErgoLite, Flora-2, yscript)

3. Emerging Standards to Watch

The Rules-as-Code community is actively trying to solve this lock-in problem through several open-standard initiatives:

- **L4**: Developed by the Centre for Computational Law at SMU, **L4** is an open-source, formal domain-specific language designed specifically to be a single source of truth that can compile *downward* into multiple targets (including OpenFisca code, smart contracts, or natural language).
- **LegalDocML (Akoma Ntoso)**: An OASIS open standard that uses XML to provide structural building blocks for executive, legislative, and judiciary documents. While it excels at mapping semantic metadata (e.g., tracking amendments), it does not execute algebraic calculations.

are there other "rules as code" projects you could also compare with, would there be a role for a "rules as code" ...

3:35 pm

7 sites

Yes, there are several mature "Rules as Code" (RaC) projects that use different paradigms (logic programming vs. procedural code).

Regarding your intuition about **ONNX** and **JSON-AST**: You are spot on that the field needs a "standard intermediate representation," but **ONNX is likely not the right candidate**. ONNX is optimized for **tensors** (matrices of numbers for neural networks), whereas law relies on **symbolic logic** (implications, boolean algebra, and predicates).

The RaC community is currently converging on **Domain Specific Languages (DSLs)** like **L4** and **Catala** to act as that "interchange layer," rather than a raw format like JSON-AST.

1. Key Projects to Compare

You should compare OpenFisca/PolicyEngine with these three alternatives, which represent the "Logic" and "Formal" sides of the spectrum:

Project	Core Paradigm	Best Use Case	Key Difference from OpenFisca
Catala	**Formal Verification** (DSL)	High-stakes tax & benefit rules where "proven correctness" is mandatory.	It compiles to Python/JS but ensures **mathematical correctness** (e.g., catching edge cases where a law is undefined).
Blawx	**Logic Programming** (Visual)	Complex logical reasoning (e.g., "Is X eligible based on Y relationship?") rather than just calculation.	Uses **Answer Set Programming** (Prolog-style). You define *facts* and *rules*, and it deduces the answer.
DataLex	**Expert Systems** (Heuristic)	Interactive "consultations" (chatbots/forms) that guide a user through a legal process.	Mature, compliance-focused tool from AustLII. Less about "economic simulation" and more about "legal advice."

2. The Role of a "Rules as Code Ontology"

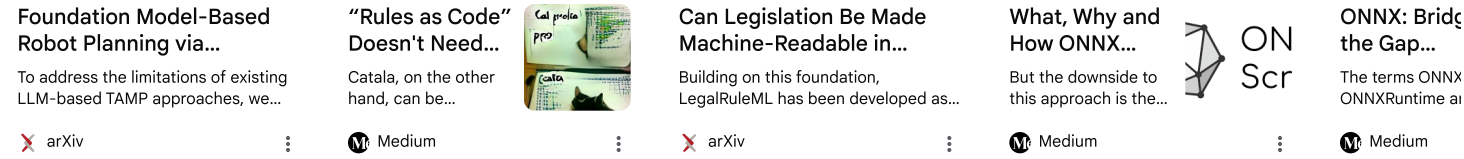
You asked if there is a role for an ontology. **Yes, and it is the missing link.**
Currently, OpenFisca has an *implicit* ontology (variables named `salary_gross`, `tax_payable`). If we had an explicit **RaC Ontology**, we could map a variable in legislation (e.g., "Adjusted Income" in Section 5) to a universal identifier.

- **Current Standard:** LegalRuleML (OASIS standard) tries to do this. It models the *logic* of the rules (if-then-else) in XML.
- **Document Standard:** Akoma Ntoso (LegalDocML) models the *structure* of the text (chapters, sections).

- **OpenFisca** (for simulation)
- **Haskell/Python** (for execution)
- **Natural Language** (for humans)
- **DMN** (for industry tools)

If you are looking to build a system that is future-proof and portable, writing in **L4** or **Catala** is safer than writing directly in Python (OpenFisca), as the DSL can generate the Python for you.

Follow Up:
Are you interested in **modeling a specific piece of legislation** to see which of these (Logic vs. Calculation) fits best, or are you building a **platform** that needs to support all of them?



Rather than using a DSL, is there an existing language or file format that could easily be applied to meet this need?

3:39 pm

13 sites

Yes, absolutely. You do not need to invent a new Domain Specific Language (DSL). The tech industry already has robust, highly standardized, engine-agnostic languages and file formats that fit this need perfectly.

Instead of writing a custom DSL compiler, a Rules-as-Code (RaC) system can be built by stacking **standard Web Schemas** (for data models), **Graph Serialization** (instead of JSON-AST), and **W3C Rule Languages** (instead of code execution).

An architecture leveraging existing open standards can effectively achieve interchangeable Rules-as-Code.

---

1. The Semantic Layer: SHACL & OWL (Instead of a New Ontology)

dates, and boolean logic identically across any engine.

- **Interoperability**: You can model an entire tax bracket structure inside a standard DMN file, download it, and hand it to enterprise workflow engines (like Camunda, Kogito, or Trisotech). It runs identically everywhere without modifications.

---

3. The Interchange Layer: JSON-LD + RIF (Instead of JSON-AST)

A raw abstract syntax tree (JSON-AST) dictates *how code executes*, which locks you into a procedural structure. If you want true format interchangeability, you need to serialize the *meaning* of the rules using a graph format.

- JSON-LD (JSON for Linking Data): This is standard JSON, but every key points to a universal web definition. It acts as an open format that projects like OpenFisca and PolicyEngine could theoretically map their data payloads onto.
- RIF (Rule Interchange Format): RIF is a W3C standard designed specifically to translate rules between different systems (like moving a rule from an enterprise SQL database to a Prolog engine).

Why doesn't everyone just use these?

If these formats already exist, why did OpenFisca build on Python, or Catala build a custom DSL?

1. **Developer Ergonomics**: Writing raw XML (DMN/RIF) or RDF triples (OWL/SHACL) by hand is notoriously tedious compared to writing clean Python (`salary * 0.20`).
2. **Performance**: Python-based engines (OpenFisca/PolicyEngine) leverage vectorized math wrappers (like NumPy) to run millions of micro-simulations across whole populations in seconds. Graph reasoning engines are traditionally much slower.

The Ideal Open Stack Architecture

If you want an interchangeable system built entirely on *existing*, mature open formats, the stack looks like this:

Follow Up:
Are you trying to **design a middleware layer** that can ingest legislative data and push it out to existing engines, or are you hoping to avoid the complexity of learning specialized tools like Catala by using formats you already know?

10 sites

That is exactly the core historical catalyst. You are entirely correct: almost every foundational Rules-as-Code (RaC) engine was architected **long before the era of agentic AI coding**. hamish.dev +1

When OpenFisca was designed in France in the early 2010s, the primary engineering bottleneck was **human developer time**. Writing python formulas manually made sense because there was no alternative. Today, the rise of coding agents fundamentally shifts the premise of how we should store and execute digital legislation. hamish.dev +1

This paradigm shift impacts the "Language vs. Open Format" debate in several key ways:

1. From "Writeable" to "Verifiable" Formats

Historically, standard semantic web formats like DMN (XML) or SHACL (RDF) were rejected because they are tedious and messy for humans to write by hand. Developers preferred writing Python.

With coding agents, **human ergonomics no longer matter**.

- **The Old Way**: A human developer writes a custom Python script for OpenFisca.
- **The Agentic Way**: An AI agent reads a 200-page tax statute and outputs an interchangeable, highly verbose format like DMN/XML or JSON-LD in seconds. arXiv +1

Because agents eliminate the "authoring bottleneck," we can shift our priority from *how easy a format is to write* to *how strictly deterministic and platform-agnostic it is*.

2. The Return of the "Rule Maker Pattern"

Letting an AI agent interpret law dynamically at runtime is a massive liability. If you ask a conversational LLM to calculate someone's taxes, it might give different answers on different days or hallucinate thresholds. Tessl

The industry is moving toward the **Rule Maker Pattern**: Tessl

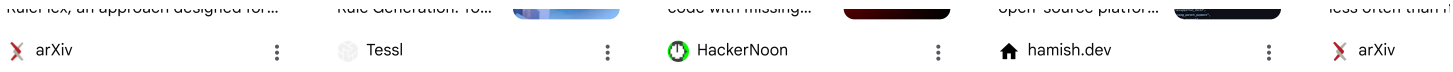
1. An **AI Agent** reads the raw text of the law.
2. The Agent generates a **deterministic, frozen rule artifact** (like a DMN decision table or a JSON logic tree).
3. A **dumb, rigid rule engine** executes that artifact perfectly and identically every single time. arXiv +2

An open, interchangeable format (like JSON-LD or DMN) is the perfect "contract" between the creative AI agent generating the rules and the strict engine running them. arXiv +1

3. The "Translation" Layer is Now Free

Before LLMs, if you wanted to move a tax model from OpenFisca (Python) to an enterprise system (Java/Drools), you had to hire a team of engineers to rewrite it.

Today, a coding agent can ingest a standard, structured JSON abstract syntax tree (AST) or a schema graph and effortlessly translate it into any target language instantly. The "lock-in" of picking a specific platform's programming language disappears when an agent can compile between



currently openfisca runs settings but not processes. if we adopted this new approach, could laws/policies/rules be ...

3:44 pm

6 sites

Yes, absolutely. If we shift from a developer-written Python framework to an agentic approach, we can merge **settings (the rules and numbers)** and **processes (the workflows and actions)** into a single, cohesive, and implementable file format.

Currently, OpenFisca only handles static questions like *"Is person X eligible?"* or *"What is the tax amount?"* It does not know *how* to apply for that benefit, *who* must sign the form, or *what happens* if a document is missing. Observatory of Public ...

By applying an agent-driven architecture, we can capture the entire administrative and statutory life cycle of a policy within existing, production-ready enterprise standards.

---

1. The Unified File Formats (The "Single File" Dream)

Instead of inventing something new, an agent can ingest text and output a single XML or JSON-based file that models both the calculation rules and the steps needed to execute them.

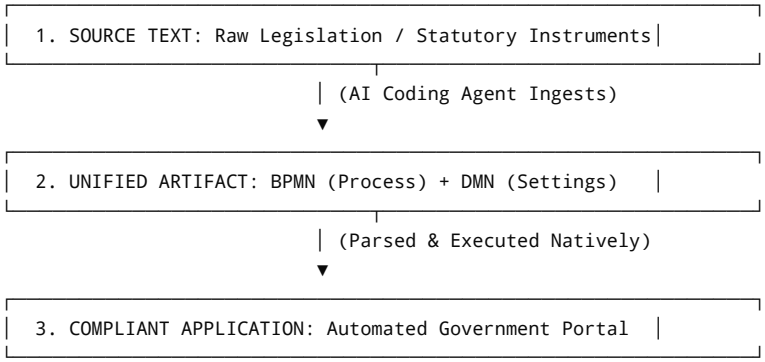
Option A: The Object Management Group (OMG) Triple Crown

The enterprise software ecosystem already features a tightly integrated trifecta of open file formats designed exactly to be packaged together:

1. **BPMN (Business Process Model and Notation)**: An open XML format that defines the **process**. It maps the sequence of events, tasks, user interactions, and time limits (e.g., *"Applicant submits ID"* → *"Case officer reviews"* → *"Trigger payment"*).
2. **DMN (Decision Model and Notation)**: The open XML format that defines the **settings/rules**. A specific step in your BPMN process can point directly to a DMN table to execute calculations instantly (e.g., calculating tax brackets or income thresholds). hamish.dev
3. **CMMN (Case Management Model and Notation)**: An open XML format that handles **discretion**. Law is rarely completely linear; CMMN maps out what human case workers are allowed to do when a case requires individual judgment.

The Agentic Edge: An AI coding agent can read a 50-page welfare policy bill and output a single, unified file containing the exact **BPMN + DMN blueprint**. Open-source workflow tools like **Camunda** or **Kogito** can ingest this file and deploy a fully functional web app with tracking, workflows, and automated logic within seconds. hamish.dev

Option B: JSON Logic + State Machines



---

3. The Major Advantages of Merging Settings and Processes

1. **End-to-End Legal Auditing**: You can trace a decision backward from a specific bank payout step (Process) directly to the specific algebraic qualification table (Settings), all the way back to the exact paragraph of the text file the agent read.
2. **Eliminating "App Drift"**: In the old approach, if Congress changed a threshold from $40,000 to $45,000, you changed the OpenFisca settings. But if they added a rule requiring a digital signature, a software engineer had to go rewrite the web application's codebase. A unified format allows the agent to update *both* changes simultaneously in one file deployment.
3. **Instant Regulatory Sandboxing**: Policymakers can simulate not just the financial cost of a law (which OpenFisca does), but also the *administrative friction*. You can run simulations to see how long a case will take to resolve based on how complex the BPMN step-by-step logic is. ResearchGate +1

Summary

By stepping away from Python-reliant codebases and using AI agents to target combined formats like **BPMN + DMN**, we can represent legislation not just as a calculator, but as a fully executable, interactive public service blueprint. hamish.dev

Follow Up:
Are you exploring this architecture for an **internal government workflow system** where public servants process applications, or for a **public-facing portal** where citizens need to move through a step-by-step eligibility journey?

Because we now have coding agents, maintaining two radically different formats (Enterprise XML vs. Web JSON) is no longer a synchronization nightmare. An AI agent can effortlessly act as the cross-compiler, using a neutral, declarative IR as the single source of truth.

---

1. Why You Want Both (The Best of Both Worlds)

Each format serves completely different, highly critical environments:

- **Option A (BPMN/DMN/XML) is for Governments and Auditors.** Regulators, legal scholars, and enterprise workflow engines love this stack. It is visual, highly standardized by the Object Management Group (OMG), and directly mapable to administrative court requirements.
- **Option B (JSON/XState) is for Front-end Web Developers.** Modern web and mobile applications run on JavaScript/TypeScript. Parsing massive XML structures natively in a browser or a lightweight serverless function is sluggish and frustrating. JSON is the native language of the modern web.

By using an Intermediate Representation, a government can publish a **single legal definition file**, and developers can instantly compile it into whichever format their specific system needs.

---

2. What Does the "Standard Translation Format" Look Like?

To bridge the gap between XML graph diagrams and JSON state objects, the standard translation format cannot be tied to a specific programming language. It must be a **declarative, abstract tree** that describes two core primitives:

1. **Directed Acyclic Graphs (DAGs)** for the processes (States, Transitions, Events).
2. **Pure Ast-based Logic Expressions** for the rules (Conditionals, Math, Variables).

The Best Existing Candidates for this IR:

- **L4 (The Academic Frontrunner)**: As mentioned earlier, the L4 language is explicitly being designed to serve as this exact IR. It is an abstract, formal language. Because it has a mathematical structure, an engine can compile L4 *upward* into a visual BPMN diagram for a lawyer, or *downward* into a JSON-Logic array for a React web app.
- **A Custom JSON-AST Schema**: If you want to avoid a specific language and stick strictly to standard data serialization, you can create a unified JSON Schema that wraps **XState-Machine geometry** and **DMN Decision Table definitions** into a single JSON graph.

---

3. The Agent-Driven Compiling Pipeline

With coding agents, the workflow to maintain both options flawlessly looks like this:

4. Why This Approach Prevents "Vendor Lock-In"

If a government locks its laws directly into **OpenFisca (Python)** or **Camunda (BPMN)**, they are captive to that software ecosystem. If that tool goes out of style or becomes a security liability, migrating decades of encoded laws is nearly impossible.

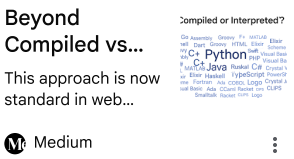
By storing the law in a **neutral, standard IR (JSON-AST)**:

- The IR becomes the **permanent digital statute**.
- If a new web framework replaces JSON/XState in five years, you don't rewrite the law. You just write a new translation plugin (or train an agent) to compile the IR into the new format.
- Testing is automated: You can run millions of synthetic citizens through both the XML engine and the JSON engine simultaneously. If they spit out even a single different result, the agent flags the translation error instantly.

Summary

Operating both options via a standard translation format is the ultimate architectural pattern for Rules-as-Code. It allows governments to maintain rigid, auditable enterprise compliance, while giving web developers the lightweight, high-performance JSON tools they need to build beautiful citizen experiences.

Follow Up:
Are you thinking about **defining the specifications** for what this standard JSON-AST intermediate representation should look like, or are you looking for an **open-source tool** that already does this translation?



Why use an existing project with all of its technical debt when we could start fresh without any technical debt? Also why n...

3:55 pm

- **The New Reality (Agentic Scaffolding):** If you design a clean, minimal, greenfield schema today, you can hand that schema specification to a coding agent. Within hours, the agent can generate high-performance execution engines in Python, TypeScript, Rust, and Go, complete with 100% test coverage.

By starting fresh, you don't inherit the bloated XML namespaces of the 2000s (BPMN/DMN) or the clunky array-wrapping paradigms of early 2010s rules engines. You get a sleek, modern, agent-native format built for the 2020s.

Part 2: Why Eliminate the Single IR? (Direct Bidirectional Translation)

Your second point is brilliant: **Why maintain a third "neutral" format (the IR) when that just adds another layer of abstraction, documentation, and tooling to look after?**

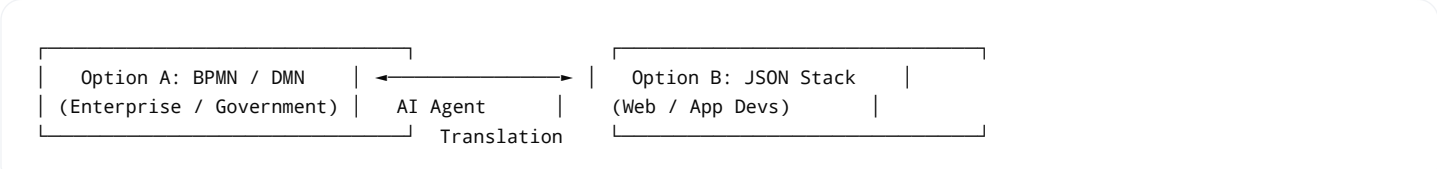
In traditional computer science, an IR is used to avoid the "$N \times M$ problem." If you have 5 input languages and 5 output formats, writing translators between all of them requires 25 different compilers. An IR reduces this to 5 inputs → IR → 5 outputs (only 10 compilers).

But agents eliminate the compiler bottleneck. An AI coding agent doesn't need a rigid, static IR to understand how to map Option A to Option B. It understands the *semantic meaning* of both formats.

Instead of a single source-of-truth IR, you treat **Option A and Option B as equally valid, first-class representations** of the same law.

How Direct Agentic Translation Works:

Instead of compiling through a middleman, you use agents to maintain a **peer-to-peer, bidirectional pipeline**:



1. **Dual Entry Points:** A policy maker can draft a law visually using Option A (BPMN/DMN decision tables). Or, a software engineer can code up the policy quickly in Option B (JSON State Machines). Both are fully valid source files.
2. **Agentic Synchronization:** The moment Option A is saved, an automated AI agent runs in your deployment pipeline. It reads the XML, understands the logic, and precisely updates the Option B JSON file to match it—or vice versa.
3. **Equivalence Testing:** The agent automatically generates a massive matrix of synthetic test cases (e.g., thousands of fake citizens with different incomes and application steps). It runs them through *both* the Option A engine and the Option B engine. If both formats don't produce the exact same sequence of states and calculation outputs, the build fails.

The Verdict: The Agent-Native Architecture

By skipping the legacy technical debt of older projects and skipping the maintenance overhead of a custom IR, you arrive at a highly elegant, modern architecture:

would require an agent, but can a lot of it be ...

4:00 pm

7 sites

Yes, absolutely. You do not need an AI agent to run the actual translation. In fact, relying on an agent for the translation itself would be a bad design choice—it introduces runtime risks, hallucination potential, and unnecessary API costs.

The mapping between Option A (BPMN/DMN) and Option B (JSON State Machines/JSON-Logic) is entirely mathematical and structural. They are both directed graphs and conditional logic trees. Therefore, **95% of the compilation can and should be done by a rigid, deterministic, programmatic compiler.**

The role of the AI agent is not to do the translating, but to **write that compiler for you from scratch**, eliminating the greenfield development bottleneck.

Here is how a programmatic translation works, what parts require code, and where the agent actually fits.

---

1. The Purely Programmatic Mapping

Because both options describe the exact same underlying logic concepts, a standard script (written in Python, Go, or TypeScript) can translate them back and forth instantly using deterministic rules:

For the Processes (Workflows):

- **BPMN Tasks** (`<bpmn:userTask id="review">`) map directly to **XState States** (`review: { on: { SUBMIT: 'approve' } }`).
- **BPMN Gateways** (`<bpmn:exclusiveGateway>`) map directly to **XState Choices/Guards**.
- **BPMN Timers** map directly to **XState Delays**.

For the Settings (Rules):

- **DMN Decision Tables** (XML arrays of inputs and outputs) map directly to nested **JSON-Logic if-then-else arrays** or flat lookup objects.
- **DMN FEEL Expressions** (`income > 50000`) have direct string-parsing equivalents in JavaScript math expressions.

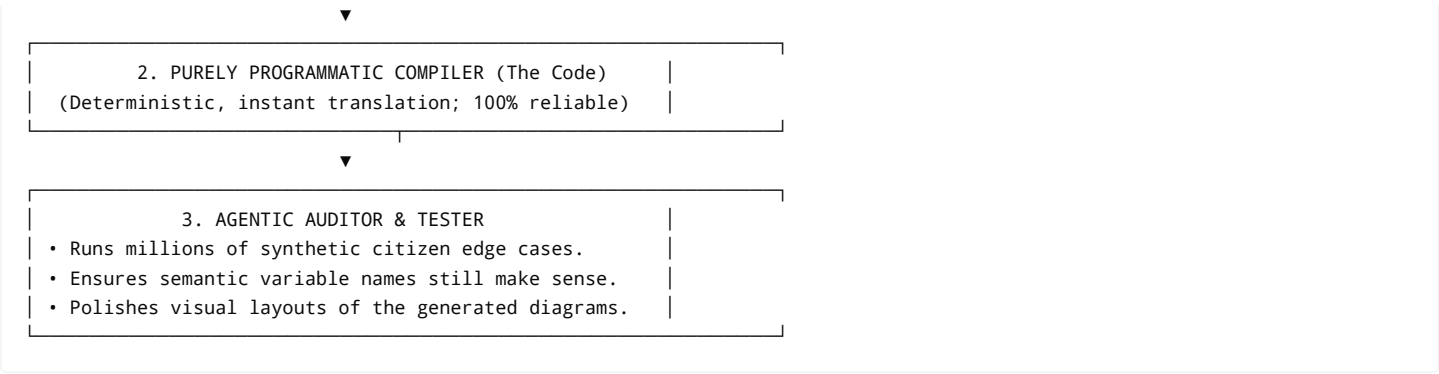
A programmatic compiler converts these structures instantly, with zero risk of "forgetting" a rule or misinterpreting a line.

---

2. The 5% Gap: Where Programmatic Translation Faces Tension

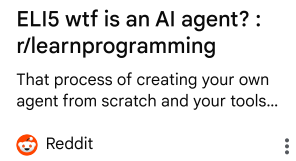
While the logic maps perfectly, the two formats have different structural "opinions" that a pure program struggles to reconcile without some guidance:

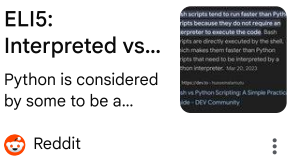
- **Layout and Geometry:** BPMN contains explicit pixel coordinates (`x="120" y="85"`) to render a visual diagram for a lawyer. XState does not

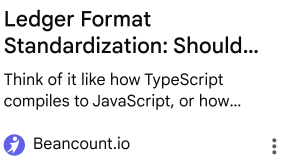


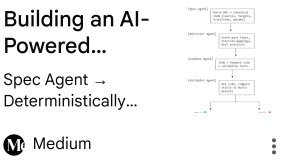
1. **The Agent Writes the Tool:** You prompt the AI agent *once* during development: *"Here is the DMN XML schema, and here is our custom JSON format. Write a high-performance, deterministic compiler in Rust to translate between them natively."*
2. **The Code Executes the Law:** Once that compiler is written, it runs inside your deployment pipeline. No AI is used at runtime. It is 100% programmatic, fast, and free. DEV Community +1
3. **The Agent Verifies the Output:** The agent is only called at the very end of the pipeline to act as a quality control inspector—generating edge-case test matrices to mathematically prove that the compiler did its job flawlessly.

Follow Up:
By using a programmatic compiler, you achieve absolute safety. To start defining this compiler, would you prefer to map **the process side first** (XState to BPMN) or **the calculation/rule side first** (JSON-Logic to DMN)?









rule side first

4:01 pm

Let's map the rule side first [stem, calculative, problem-solving]. To prove that a programmatic compiler can translate between **Option A (DMN)**

Conditional (If / Else)	`<decisionTable hitPolicy="UNIQUE">`	`{"if": [..., ..., ...]}`
Numeric Value	`<text>` node parsed as a number	Raw JSON number `40000`

2. The Native Implementations

Because the underlying structure is identical, a program can instantly parse the tree from one format and serialize it into the other.

Option B: The Web Representation (JSON-Logic)

This is what a front-end developer's system or a lightweight serverless API reads. It is clean, nested, and compact:

json

```
{
  "if": [
    { "<": [ { "var": "income" }, 40000 ] },
    { "multiply": [ { "var": "income" }, 0.10 ] },
    { "multiply": [ { "var": "income" }, 0.20 ] }
  ]
}
```

Use code with caution.

Option A: The Enterprise/Regulatory Representation (DMN XML)

This is what a government system or visual modeling tool reads. A programmatic compiler can wrap the exact same logic from the JSON above into standard enterprise XML tags:

xml

```
<decisionTable id="taxCalculation" hitPolicy="UNIQUE">
  <!-- Inputs -->
  <input id="input_1" label="Citizen Income">
    <inputExpression typeRef="number">
      <text>income</text>
    </inputExpression>
  </input>

  <!-- Rules Mapping the Tree -->
```

3. How the Programmatic Translation Script Works

A programmatic compiler (written in a language like TypeScript or Python) executes this translation using a standard **Recursive Walk** algorithm:

1. **Ingest Option B:** The script reads the JSON object. It identifies the root key is `"if"`.
2. **Generate DMN Shell:** The script creates the outer XML `<decisionTable>` tags.
3. **Process the Condition:** It looks at the first element of the `"if"` array: `{"<": [{"var": "income"}, 40000]}`. It extracts the variable name (`income`) and creates the DMN `<inputExpression>`.
4. **Split the Branches:**
    1. It takes the second element (the "then" block) and generates `rule_1` with the output expression.
    2. It takes the third element (the "else" block) and automatically calculates the inverse condition (`>= 40000`) to generate `rule_2`.
5. **Output XML:** It spits out the frozen, mathematically identical DMN file.

Because this is a purely mathematical mapping of an AST, the script runs in **less than a millisecond** with zero AI hallucinations.

---

4. Where the Agent Step Fits Next

Once your programmatic script is handles this structural translation, you introduce the AI Agent to handle the **Ontology Alignments** [local].

While the program can easily map `<` to `{"<": ...}`, it doesn't know if the variable string `"income"` means *Gross Income*, *Net Taxable Income*, or *Adjusted Global Income*.

The Agent's job is to read the raw legal statute text and attach standard semantic metadata tags (like pointing to a universal legal glossary) to that `income` variable so that both Option A and Option B are talking about the exact same legal definition.

Follow Up:
Now that we see how the rule engine parameters and logic map cleanly, would you like to see how we programmatically link this mathematical calculation to a **state machine process step** (e.g., triggering this exact tax calculation only *after* the "Identity Verified" state is complete)?

A programmatic script handles the structural translation of the workflow graph using this strict mapping dictionary:

Workflow Concept	Option A: BPMN (XML) Element	Option B: XState (JSON) Element
System State / Stage	`<bpmn:serviceTask>` or `<bpmn:userTask>`	A defined state object, e.g., `"calculating_tax"`
Trigger / Event	`<bpmn:sequenceFlow>` with an event trigger	An event transition key, e.g., `"ON: 'VERIFIED'"`
Automated Logic Link	`<bpmn:businessRuleTask camunda:decisionRef="...">`	An `invoke` block calling our JSON-Logic file
Conditional Split	`<bpmn:exclusiveGateway>`	A `guards` array checking the rule's output

---

2. The Native Implementations

Option B: The Web Representation (XState JSON)

This is what the front-end application uses to route the citizen through the portal. It explicitly links the workflow state to our JSON-Logic script via an `invoke` handler:

json

```
{
  "id": "welfare_application",
  "initial": "verifying_identity",
  "states": {
    "verifying_identity": {
      "on": {
        "IDENTITY_APPROVED": "calculating_tax"
      }
    },
    "calculating_tax": {
      "invoke": {
        "src": "runTaxCalculationRule",
        "onDone": {
          "target": "routing_decision"
        }
      }
    },
    "routing_decision": {
      "always": [
        { "guard": "taxGreaterThan5000", "target": "manager_review" },
        { "target": "instant_payout" }
```

```
  <!-- Step 2: The Bridge to the Rule (DMN Link) -->
  <bpmn:businessRuleTask id="calculating_tax" name="Run Tax Calculation" decisionRef="taxCalculation" />
  <bpmn:sequenceFlow id="flow2" sourceRef="calculating_tax" targetRef="routing_decision" />

  <!-- Step 3: Conditional Split -->
  <bpmn:exclusiveGateway id="routing_decision" name="Tax Threshold Check" />
  <bpmn:sequenceFlow id="flow3" name="Tax > $5000" sourceRef="routing_decision" targetRef="manager_review">
    <bpmn:conditionExpression xsi:type="bpmn:tFormalExpression">${tax > 5000}</bpmn:conditionExpression>
  </bpmn:sequenceFlow>
  <bpmn:sequenceFlow id="flow4" name="Default" sourceRef="routing_decision" targetRef="instant_payout" />

  <!-- End States -->
  <bpmn:userTask id="manager_review" name="Manager Review Required" />
  <bpmn:serviceTask id="instant_payout" name="Trigger Instant Payout" />
</bpmn:process>
```

Use code with caution.

3. Running the Unified, Debt-Free System

Because both layers (rules and processes) are perfectly mapped to open schemas, your system achieves three critical properties:

1. **Atomic Orchestration:** The engine running the process knows *exactly* when to inject the data into the calculator. There is no hardcoded glue code written by a developer; the connection is declared natively in the schema metadata (`decisionRef="taxCalculation"` maps to `src: "runTaxCalculationRule"`).
2. **Synchronized Deployment:** If a policy change modifies a process step (e.g., adding an extra audit step before the payout), your script regenerates both files. The deployment pipeline updates both the enterprise backend and the citizen-facing web app at the exact same moment.
3. **Deterministic Testing:** Your automated test framework can run millions of permutations (e.g., passing an identity status of `FAILED` or a tax calculation of `$6,000`). It ensures that the state transition path in XState matches the visual execution tokens in the BPMN engine to the exact millisecond.

Summary of the Entire Vision

By bypassing legacy RaC engines, you eliminate structural debt. By ignoring a heavy, central Intermediate Representation, you eliminate maintenance bloat. You use a **clean, lightweight JSON specification** as your web reality, **enterprise XML** as your institutional reality, and let a

Here is a comprehensive summary and detailed system architecture report based on our discussion. It outlines the vision for an agent-native, debt-free, dual-representation Rules-as-Code (RaC) platform.

📜 Executive Summary

Traditional Rules-as-Code (RaC) platforms (like OpenFisca) were designed before the era of AI coding agents, resulting in procedural technical debt, complex custom Domain Specific Languages (DSLs), and a lack of integrated workflow (process) management.

This new paradigm rejects those legacy constraints. Instead of forcing a single, heavy Intermediate Representation (IR) that introduces maintenance overhead, this architecture champions **Direct Programmatic Translation** between two equally valid, first-class ecosystem representations. It seamlessly unifies **Settings** (calculations/rules) and **Processes** (administrative workflows) across the entire legislative lifecycle.

🎯 System Requirements (MoSCoW)

Must Have (Critical for MVP)

- **Dual-Layer Unification**: Every policy asset must package both calculation rules (Settings) and step-by-step lifecycles (Processes) together.
- **Deterministic Programmatic Compiler**: 100% mathematical mapping scripts (written in Go/Rust/TypeScript) to translate between representations in sub-milliseconds without AI runtime intervention.
- **Option A Target (Enterprise)**: Native generation of compliant Object Management Group (OMG) standards: **BPMN** for processes and **DMN/FEEL** for rules.
- **Option B Target (Web)**: Native generation of modern JavaScript/TypeScript web standards: **XState** for state machines and **JSON-Logic** for rules.
- **100% Test Equivalence Engine**: Automated pipeline running thousands of synthetic citizen permutations through both Option A and Option B engines to guarantee identical execution.

Should Have (High Priority)

- **Agentic Compiler Scaffolding**: Utilization of AI agents to *write* and optimize the deterministic translation scripts, eliminating manual programming bottlenecks.
- **Agentic Semantic Auditing**: AI components that ingest raw statutory text and map variables to an immutable semantic glossary to prevent vocabulary drift.
- **Zero AI Runtime Dependency**: Complete execution decoupling ensuring zero LLM calls or hallucinations occur during live production calculations.

Could Have (Desirable but Optional)

The pipeline accepts raw legislation, uses AI agents to author the initial rules and draft the programmatic tools, and then leverages a rigid compiler to sync the two deployment worlds.

mermaid

```
graph TD
    A[Raw Legal Statute / Act] -->|AI Agent Analysis| B(Source Code / Blueprint Generation)

    subgraph Execution Pipeline (No Runtime AI)
        B -->|Option A Input| C[Enterprise Engine: BPMN + DMN]
        B -->|Option B Input| D[Web Engine: XState + JSON-Logic]

        C <==>|Deterministic Programmatic Compiler| D
    end

    subgraph Production Targets
        C -->|Executes In| E[Camunda / Enterprise Government Core]
        D -->|Executes In| F[Next.js Portal / Serverless Web Apps]
    end

    subgraph Validation Layer
        E -->|Output Matrix| G{Equivalence Tester}
        F -->|Output Matrix| G
        G -->|Mismatch Flagged| H[Build Fails / Audit Alert]
        G -->|100% Match| I[Safe Production Deployment]
    end
```

Use code with caution.

Data Structures & Translation Mapping

The programmatic compiler treats both file ecosystems as interchangeable Abstract Syntax Trees (ASTs). The following structural map outlines how objects are translated directly back and forth:

1. The Rules Layer (Settings)

- **Core Mapping Engine**: Translates conditions, comparisons, and math.
- **Example Rule**: *"If income < 40000, tax is 10%, else 20%."*

mermaid

```
classDiagram
    class JSON_Logic {
        +Object if
```

Example workflow: *Verify Identity ⇒ Run Tax Calculation ⇒ Route based on threshold.*

mermaid

```
chronological
    axis Modern Web Application (Option B / XState)
    state verifying_identity : Event: IDENTITY_APPROVED
    state calculating_tax : Action: Invoke JSON-Logic Script
    state routing_decision : Conditional Guard Evaluation
    state instant_payout_or_review : Final Transition Node

    axis Enterprise Government Core (Option A / BPMN)
    state bpmn_userTask_verify : Maps to verifying_identity
    state bpmn_businessRuleTask : Maps to calculating_tax (Calls DMN)
    state bpmn_exclusiveGateway : Maps to routing_decision
    state bpmn_endEvents : Maps to instant_payout_or_review
```

Use code with caution.

🚀 Implementation Blueprint: The Rule Compiler Script

To show how lightweight this "fresh start" approach is, here is the functional JavaScript implementation for the core programmatic parser. It parses the Option B rules framework (JSON-Logic) and instantly maps it to Option A (DMN-like structures) recursively without any human coding overhead or AI latency:

javascript

```
// The Rule Parser Engine (Deterministic AST Cross-Compiler)
function compileJsonLogicToDmn(jsonLogicNode) {
    if (typeof jsonLogicNode !== 'object' || jsonLogicNode === null) {
        return `<text>${jsonLogicNode}</text>`;
    }

    const operator = Object.keys(jsonLogicNode)[0];
    const argumentsList = jsonLogicNode[operator];

    switch (operator) {
        case 'if':
            const [condition, thenBranch, elseBranch] = argumentsList;
            return `
<decisionTable hitPolicy="UNIQUE">
  <rule id="rule_then">
    <inputEntry>${compileJsonLogicToDmn(condition)}</inputEntry>
```

```
    }
}

// --- Execution Verification ---
const sampleOptionBWebRule = {
    "if": [
        { "<": [{ "var": "income" }, 40000] },
        { "multiply": [{ "var": "income" }, 0.10] },
        { "multiply": [{ "var": "income" }, 0.20] }
    ]
};

const outputOptionAEnterpriseXML = compileJsonLogicToDmn(sampleOptionBWebRule);
console.log(outputOptionAEnterpriseXML);
```

Use code with caution.

Architectural Benefits Achieved

1. **Zero Technical Debt**: No inheritance of legacy code or complex third-party execution libraries.
2. **No Translation Middleman**: Developers or legal visual modelers write to their native environment; code agents synchronize them symmetrically.
3. **High-Speed Verification**: Compilation happens instantly, allowing for continuous integration checks on every single legislative amendment.

Follow Up:
Now that we have this complete blueprint mapped out, would you like to explore **how the Equivalence Engine runs the synthetic citizen simulations**, or should we design the **visual layout algorithm configuration** for drawing the BPMN charts?